QA Concept for CHESS MANIA!

Quality Assurance is a broad term, explained on the Google Testing Blog as “the continuous and consistent improvement and maintenance of process that enables the QC job”. As follows from the definition, QA focuses more on organisational aspects of quality management and monitoring the consistency of the production process.

There are a wide variety of ‘best practices’ for QA currently adopted in software development, we have decided to implement the following procedures in working on our project:

1. Communication

Our Team uses Zoom to attend meetings where the very next logical development steps are decided and tasks are assigned to each one of us. The Gantt Project file with all the milestone achievements is used as a compass to guide this decision. We also update our Gantt Project to ensure that everything is documented and up to date. After assignment a Trello chart is created where we can self-assign a task that we are working on. This is to ensure transparency in the working process and avoid unpleasant surprises.

During the exercise sessions and the lecture, we meet up on Zoom to work on the tasks that need both our attendance. If a synchronous session is not an option we use Slack or WhatsApp for ongoing communication about tasks and issues and for sharing relevant resources.

After a Milestone is reached, the process begins anew.

2. Explaining the Code / Code Review

After a task is completed, we regularly explain our implementation to each other. This is to ensure that there are no obvious mistakes left in the code and also to ensure that team members are up to date on the status of the project. This process also doubles as a Code Review where the work is checked for style (we are using the Google Style Guide) and content.

3. Documenting the Code

All code is extensively documented in English using Javadoc in order for us to understand each other's code.

4. Bug Report and Debugging

Every time a bug is found, we immediately create a new Bug Report containing all important information and post it on our communication platform. This procedure turned out to be very helpful, as we were able to fix numerous issues. After a Bug Report is sent out, the responsible group member can recreate the issue and start to debug. We use the Debugger in IntelliJ to isolate the parts of the code that are causing problems. After finding the problems we are able to correct them.

5. Logging

We agreed on using the logging framework log4j2 to log errors in the hope that this will allow more constructive error handling. We will also be tracking the amount of Logging-Statements in our project.

6. Testing

We test our code (Black Box & White Box) as soon as possible to make sure it functions as expected so that any detected issues are fixed and do not snowball. This is crucial because the cost of an error tends to increase throughout the software development process.

7. Unit Tests

We will start to implement JUnit-Tests to ensure that our code is behaving according to its intended purpose. We will track their coverage with the IntelliJ plugin “Jacoco” and we will use JUnit-Tests to test all fundamental actions related to the game logic as for example the different moves that a chess piece can and can't do and also their ability to capture opponent pieces and recognize their teammates pieces.

8. Metrics

We are using the IntelliJ Metrics Reloaded Plugin to monitor the quantitative features of our source code. The three metrics that we will be monitoring during our project are: Lines of code metrics, Javadoc coverage metrics and Complexity metrics.

The lines of code metric returns the total amount of lines of code in our project and also the average amount of lines of code per package. The javadoc coverage metric returns useful values like javadoc class coverage or javadoc method coverage which indicate how much of the code is documented using javadoc. The complexity metric is used to indicate the complexity of a program. We will be looking only at the Cyclomatic complexity. All these values are calculated by the Metrics Reloaded Plugin and transferred to a table.

Milestone 3 - Metrics results:
The data can be found in the metrics.pdf file

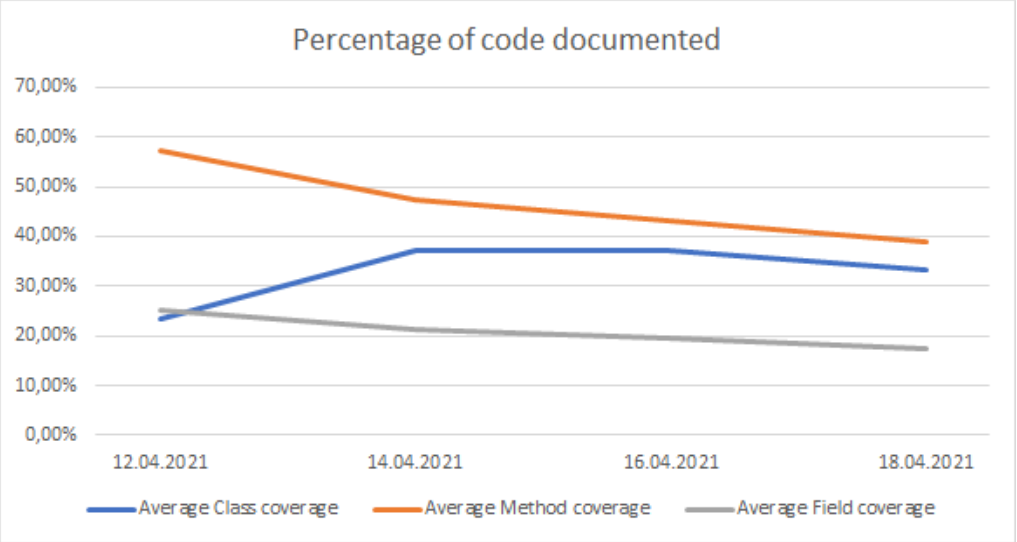


Figure 1: The percentage of documented code. The x-axis shows the tracked dates and the y-axis shows the percentage of documented classes, methods and fields.

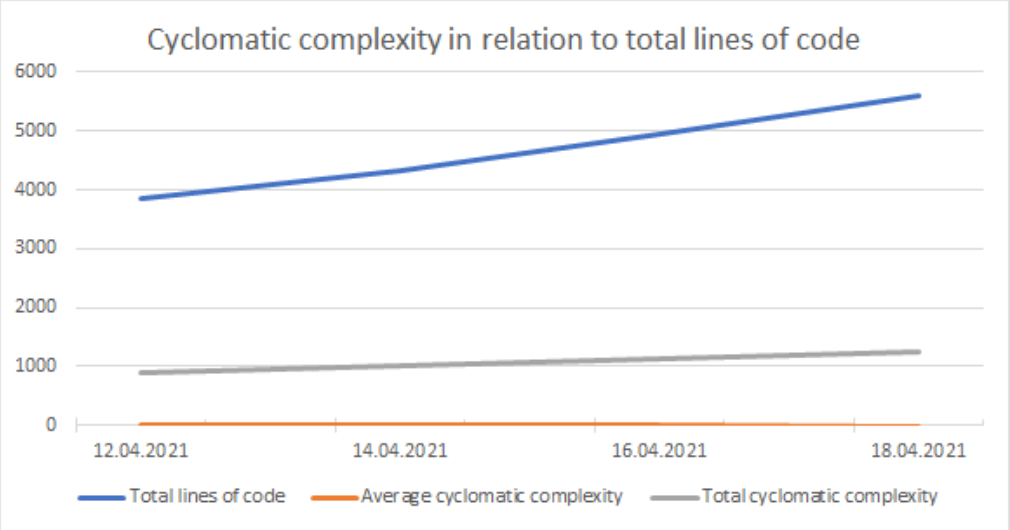


Figure 2: The cyclomatic complexity in relation to the total lines of code.

We haven't yet created any Unit-Tests. For that reason our Code Coverage is 0.

We have very recently added the library log4j2 to our project therefore the current amount of Logging-Statements in our project is 4.